

Documentation Technique - EasySave v1.0

Application Console

1. Vue d'ensemble technique

EasySave est une application console de sauvegarde developpee en C# .NET 8.0. Elle separe la logique metier dans EasySave et la journalisation dans EasyLog. L'application gere jusqu'a 5 taches, prend en charge les modes Full et Differential, maintient l'etat en temps reel et ecrit un journal quotidien au format JSON.

Le projet repose sur une execution sequentielle des jobs, une persistance locale simple et un affichage bilingue. L'objectif technique est de garder une architecture lisible, assez modulaire pour faire evoluer les sauvegardes, tout en restant compatible avec un usage console direct depuis PowerShell.

2. Architecture technique

2.1. MVVM et composants principaux

Program.cs compose les services. ApplicationViewModel charge les jobs, prepare l'affichage et declenche les sauvegardes. BackupController orchestre l'execution sequentielle. BackupService valide les chemins, filtre les fichiers et pilote la copie. StateService, BackupJobRegistry, BackupHistoryService et LoggerService gerent la persistance runtime.

La separation des responsabilites est nette : la vue console se limite au rendu, le ViewModel centralise les decisions d'interface, le controller relaie l'intention metier et les services encapsulent les acces disque, les calculs de progression et les ecritures JSON.

L'architecture suit une variante MVVM adaptee a une application console : ConsoleApplicationView joue le role de View, ApplicationViewModel porte l'etat d'affichage et les commandes, et les modeles metier restent separes dans BackupJob, BackupState, BackupResult et LogEntry. Ce n'est pas un MVVM graphique avec data binding, mais la responsabilite de presentation est bien isolee de la logique metier.

2.2. Technologies utilisees

Le projet est developpe en C# sur .NET 8 avec le SDK Microsoft.NET.Sdk. EasySave est un executable console, EasyLog est une bibliotheque separee referencee par ProjectReference, et EasySave.Tests couvre le comportement avec xUnit, Microsoft.NET.Test.Sdk, xunit.runner.visualstudio et coverlet.collector.

Pour la persistance, l'application utilise System.Text.Json et JsonSerializerConverter. Pour le systeme de fichiers, elle s'appuie sur Directory, File, Path et FileInfo. Les durees de traitement sont mesurees avec Stopwatch, et l'usage vise principalement une execution PowerShell ou terminal Windows.

2.3. Flux d'execution

Sequence principale :

```
Commande utilisateur -> ArgumentParser -> ApplicationViewModel -> BackupController
-> BackupService -> StateService / LoggerService / BackupHistoryService
```

Pour une sauvegarde differentielle, BackupService recopie un fichier s'il n'existe pas encore dans la cible ou si sa date UTC est plus recente que le dernier Full reussi.

Pendant l'execution, StateService ecrit l'etat courant apres chaque etape significative. LoggerService ajoute une entree pour chaque copie, creation de dossier ou erreur. En fin de traitement, le statut passe a Finished ou Error, puis BackupHistoryService memorise le dernier Full reussi quand le job est complet et sans echec.

2.4. Persistance des donnees

Les fichiers runtime principaux sont :

```
jobs.json : configuration persistante des 5 slots de sauvegarde
state.json : etat courant et progression de tous les jobs
backup-history.json : date du dernier Full reussi par job
yyyy-MM-dd.json : journal quotidien des copies et erreurs
storage-settings.json : dossier runtime et langue memorisee
```

La langue suit l'ordre de priorite suivant : EASYSAVE_LANGUAGE, langue enregistree via --lang, puis langue du systeme.

Cette persistance permet de relancer l'application sans perdre la configuration des jobs, le dernier historique utile aux sauvegardes differentielles ni l'etat observe par un utilisateur pendant l'execution.

2.5. Explication des diagrammes UML

Le diagramme de cas d'utilisation represente les actions console disponibles : affichage, aide, configuration, selection de jobs, langue et dossier runtime. Il correspond directement aux commandes parsees par ArgumentParser puis interpretees par ApplicationViewModel.

Le diagramme de classes doit montrer la presentation (ConsoleApplicationView, ApplicationViewModel), l'orchestration (BackupController, IBackupService, BackupService) et la persistance (BackupJobRegistry, StateService, LoggerService, BackupHistoryService, RuntimeStoragePaths). Les modeles BackupJob, BackupState, BackupResult et LogEntry transportent les donnees entre ces blocs.

Le diagramme de sequence illustre la chaine Commande -> ViewModel -> Controller -> Service -> JSON/logs. Il doit aussi mettre en evidence que les ecritures dans state.json et dans le log quotidien sont intercalees avec la copie des fichiers, pas realisees seulement a la fin.

2.6. Analyse des commandes

ArgumentParser supporte plusieurs formes de selection : un job unique, une plage comme 1-3, ou une liste personnalisee comme 1;3;5. Il gere aussi les commandes --help, --configure, --storage-dir et --lang, puis retourne un CliCommand que le ViewModel interprete sans melanger cette logique au rendu console.

Les validations sont faites tot : numero de job entre 1 et 5, format de plage correct, ordre debut/fin valide et code de langue limite a fr ou en. Cette etape evite d'entrer dans la logique de sauvegarde avec une commande incoherente.

2.7. Gestion du stockage runtime

RuntimeStoragePaths centralise tous les chemins techniques du projet. Par default, l'application utilise AppContext.BaseDirectory, mais elle peut basculer vers un autre dossier au moyen de storage-settings.json et de l'option --storage-dir. Les chemins relatifs sont resolus depuis le dossier de base puis normalises.

Cette approche supprime les chemins hardcodes et permet de deplacer en bloc jobs.json, state.json, backup-history.json et les logs quotidiens sans modifier le reste de l'application.

2.8. Points techniques notables

BackupJobRegistry normalise toujours cinq jobs. StateService maintient aussi les jobs inactifs dans state.json. Les erreurs de validation ou de copie alimentent a la fois l'etat et le log quotidien. EasyLog reste isole pour conserver une DLL de journalisation reutilisable.

Autre point important : la progression ne repose pas seulement sur le nombre de fichiers mais aussi sur le volume total en octets. Cela permet d'exposer un avancement plus fidele, surtout lorsque les fichiers copies ont des tailles tres differentes.

Enfin, StateService resynchronise toujours le snapshot avec la configuration des jobs. Un job supprime de jobs.json disparaît de l'etat, tandis qu'un nouveau job est ajoute comme Inactive avant toute execution.

Le projet inclut aussi une couche de tests separee. Cela renforce la maintenabilite du parsing de commandes, des services et des comportements critiques avant une evolution vers des versions plus riches.